

TP 6 : Introduction à Bison & l'Analyse Syntaxique

Année universitaire : 2024 – 2025

Enseignante : Aida Lahouij

1. Utilisation de Bison

Bison est un générateur d'analyseurs pour les grammaires hors-contexte. Il est utilisé pour reconnaître les phrases d'un langage.

Bison génère du code source en C pour l'analyseur syntaxique à partir d'un fichier de spécification fourni (par exemple : monfichier.y). Le fichier généré aura le même nom que le fichier de spécification, mais avec l'extension .tab.c.

Par exemple, si votre fichier Bison s'appelle monfichier.y, le fichier C généré sera monfichier.tab.c.

Commande pour générer l'analyseur syntaxique :

```
bison monfichier.y
```

Ensuite, compilez le fichier source généré pour produire l'exécutable de l'analyseur :

```
gcc monfichier.tab.c -o monfichier.exe
```

Si gcc produit une erreur liée à la fonction manquante yywrap(), compilez en ajoutant l'option de liaison -ly.

Il est important de noter que Bison seul ne suffit pas pour construire un compilateur complet. Un analyseur lexical (ex. avec Flex), des actions sémantiques et des modules de génération de code doivent être ajoutés.

2. Format du fichier de spécification

Le format d'un fichier de spécifications est le suivant :

```

%{
Déclarations en C : variables, constantes, fichiers d'en-tête, etc.
%}

Déclarations de jetons lexicaux

Déclarations de précédence et de types

%%

Règles de production de la grammaire et actions associées :

NonTerminal : liste_de_symboles { action C 1 }
              | liste_de_symboles { action C 2 }
              | ...
              ;

%%

Routines C et fonction main

```

- a) les symboles sont représentés par des identificateurs
 - Terminaux : lexèmes provenant de l'analyse lexicale
 - Non-terminaux : symboles internes à l'analyseur syntaxique
 - b) le code C est optionnel et est exécuté lorsque la règle est réduite
- Pour exécuter l'analyseur syntaxique généré, il faut lui fournir comme entrée un flot d'unités lexicales. Ce flot est l'output obtenu après l'exécution de l'analyseur lexical.

3. Coordination entre les analyseurs lexical et syntaxique

Pour transmettre la sortie de l'analyseur lexical à l'analyseur syntaxique, une coordination entre les deux est nécessaire, comme illustré par la flèche rouge sur le schéma.

Étapes d'intégration :

- Définir les jetons dans l'analyseur syntaxique à l'aide de %token.

- Inclure le fichier d' en-tête généré par Bison dans le fichier de spécification Flex.

Dans le fichier Flex (monfichier.l), inclure dans la section des déclarations :

```
#include "monfichier.tab.h"

extern int yyval;
```

Pour que Bison génère un fichier .h en plus du code source en C, il faut utiliser l'option -d. La compilation s'effectue par conséquent comme suit :

```
>> bison -d monfichier.y

>> flex monfichier.l

>> gcc -o parser lex.yy.c monfichier.tab.c
```

Ou bien:

```
>> gcc -o parser lex.yy.c monfichier.tab.c main.c
```

Exercices:

Exercice1:

1. Soit la spécification Flex suivante

```
/*fichier tp6.l */
%{
#include "tp6.tab.h"
#include<stdlib.h>
extern int yylval;
int nblignes=1;
}%
%option noyywrap
chiffre [0-9]
entier {chiffre}+
espace [ \t]
%%
somme return (SOMME);
produit return (PRODUIT);
```

```

\n nblignes++;
[,\\.] return (yytext[0]);
{entier} {yylval=atoi(yytext); return NOMBRE;}
{espace}+ ;
"$" return (FIN);
. printf("ERREUR ligne %d: %c inconnu\n", nblignes,
yytext[0]);
%%

```

2. Soit la spécification Bison suivante

```

/*fichier tp6.y */
%{
#include<stdio.h>
#include<stdlib.h>
%}
%token SOMME
%token PRODUIT
%token NOMBRE
%token FIN
%%
texte: liste texte
| FIN {printf("Merci et a bientot\n"); YYACCEPT;}
;
liste: SOMME sentiers '.' {printf("la somme est
%d\n", $2);}
| PRODUIT pentiers '.' {printf("le produit est %d\n", $2);}
;
sentiers: sentiers ',' NOMBRE {$$=$1+yylval;}
| NOMBRE {$$=yylval;}
;
pentiers: pentiers ',' NOMBRE {$$=$1*yylval;}
| NOMBRE {$$=yylval;}
;
%%
int yyerror(void)
{ fprintf(stderr, "erreur de syntaxe\n");
return 1;}
void main(void)
{
yyparse();
}

```

3. Ecrire et compiler les spécifications ci-dessus

4. Quel est le rôle de cet analyseur ? expliquer

5. Tester cet analyseur avec des exemples de votre choix

Exercice2:

Écrire un analyseur avec **Bison** qui valide si une chaîne est une **expression bien parenthésée**. Par exemple :

Entrée Résultat

() valide

((())) valide

((invalide

()))(invalide

Exercice3:

Écrire un analyseur syntaxique (avec **Flex & Bison**) qui reconnaît et valide un **fichier JSON simplifié** contenant uniquement :

- des **objets JSON**
- des **paires clé-valeur**
- des **chaînes de caractères**

```
{  
  
  "clé1": "valeur1",  
  
  "clé2": "valeur2",  
  
  ...  
  
}
```

Exemple d'entrée :

```
{  
  "name": "Alice",  
  "age": "25",  
  "city": "Paris"  
}
```

Règles syntaxiques

1. Un objet JSON commence par { et se termine par }.
2. Il contient **0 ou plusieurs** paires de la forme "clé" : "valeur".
3. Les paires doivent être séparées par des virgules ,.
4. Les clés et valeurs sont des **chaînes entre guillemets doubles** "...".